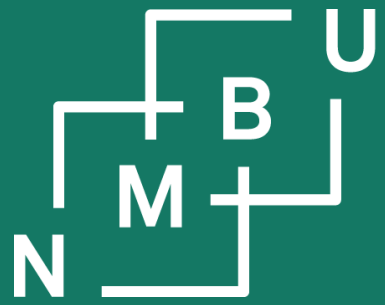
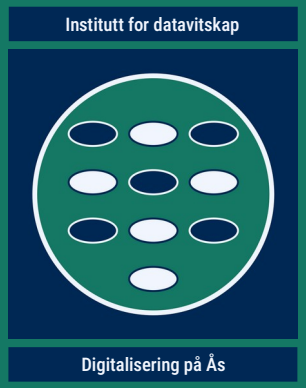


Noregs miljø- og biovitskaplege universitet



INF205
Ressurseffektiv programmering

2 Minnehandtering

2.1 Kallstakken og haugen
2.2 Vanlege minnehandteringsfeil
2.3 Containerklasser (kopiering og flytting)
2.4 Templat og abstrakte klasser

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

Problemliste 1

INF205

25. februar 2026

Problem 1.1: Storleik på dataelement

Skal diskuterast av **gruppe 17** og **gruppe 4**.

Bruk sizeof til å finne ut kor store dei forskjellige fundamentale datatypane, deriblant short, int, long og long long mm., er på dykkar system.

Verifiser i eitt tilfelle at resultatet frå sizeof stemmer overeins med avstanden i minnet mellom ein peikar på den respektive type objekt og den verdien peikaren får ved inkrementering.

Problem 1.2: Bytevis representasjon av tal

Skal diskuterast av **gruppe 21** og **gruppe 1**.

Lagre talverdien –12 som høvesvis short, int, long og long long.

Vis innhaldet i heksadesimalnotasjon (0x...).

I heksnotasjon er det to siffer som representerer éin byte.

Forklar resultata dykkar.

Problem 1.3: Fil-input/-output

Skal diskuterast av **gruppe 9** og **gruppe 11**.

Rediger eksempelet io-operator-overloading.zip slik at, i staden for ein std::stringstream, ei fil blir brukt til å mellomlagre det serialiserte objektinnhaldet.

Problem 1.4: Fil-input/-output (#2)

Skal diskuterast av **gruppe 23** og **gruppe 2**.

Implementer filinput for BookIndex, slik at den kan ta inn ei fil med innhald som:

```
3
3 1 50
2 4 40
3 4 60
```

Talet tre i fyrste line betyr at det er tre objekt.

Problem 1.5: Gjengje argumentlista utan duplikat

Skal diskuterast av **gruppe 15** og **gruppe 7**.

No vil vi oppnå det same som ved komline-argsett.cpp, dvs. at koden går gjennom kommandolineargumenta og gjengjev dei, men kvart argument maksimalt éin gong.

Men eksempelkoden sorterte argumentlista i tillegg av di at std::set er ein sortert datastruktur. Gjer det den gongen utan å omsortere argumenta, og hald fast på den opphavlege rekkjefylgja.

Problem 1.6: Bruk av std::array

Skal diskuterast av **gruppe 18** og **gruppe 10**.

Gjer det same som i string-argument-passing.cpp, men bruk STL-datastrukturen std::array<char, 7>.

Implementer både verdioverføring og referanseoverføring.

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

2 Minnehandtering

2.1 Kallstakken og haugen

2.2 Vanlege feil

2.3 Containerklasser

2.4 Templat of abstrakte klasser

INF205

25. februar 2026

Ordliste

abstract class, interface	abstrakt klasse, grensesnitt
linked data structure	lenkja datastruktur
linked list	lenkja liste
overriding	overstyring
taxonomy, class hierarchy	taksonomi, klassehierarki
virtual method	virtuell metode

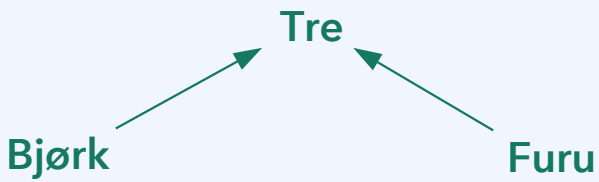
Taksonomi (klassehierarki) og virtuelle metodar

Ved bruk av ein taksonomi, med superklasser og **subklasser som er avleidde av superklassen**, er det **arv**: Objekt som instansierer subklassen er òg instansar av superklassen, som dimed får alle dei medlemene som er definert både for super- og subklassen.

Eksempelfiler:

utan-virtuell-metode.cpp

med-virtuell-metode.cpp



Men **kven tek førerang** ved overlasting, når ein metode er definert både i den høgare og i den avleidde klassen? Koss heng det saman med **statisk typing**?

Eksempelet innfører bruk av **virtuelle metodar** og **overstyring** (overriding).

Taxonomy (class hierarchy)

Classes can stand in a hierarchical relationship: A more general superclass and its more specific subclass (also, “derived class” or “child”).

An object of the subclass then (automatically) is **also an object of the superclass**; it has all the members defined in its class definition, but also **inherits the members defined for the superclass**, to which it also belongs.

Example file: literature-indices.zip

Class hierarchy implementation in C++

Classes can stand in a hierarchical relationship: A more general superclass and its more specific subclass (also, "derived class" or "child").

An object of the subclass then (automatically) is **also an object of the superclass**; it has all the members defined in its class definition, but also **inherits the members defined for the superclass**, to which it also belongs.

Example file: literature-indices.zip

```
class LiteratureIndex {
public:
  virtual int next_page();
  …
private:
  int year = 0;
  …
};
```

```
class JournalArticleIndex: public LiteratureIndex {
public:
  int next_page();
  …
private:
  int volume = 0;
  …
};
```

JournalArticleIndex has the property **volume**, but it also **inherits the property year**.

It can override the **next_page** method definition from its superclass, because it is **virtual**.

C++ standard template library

The standard template library (STL) provides typical **container** data structures. They are **templates**: They can contain any type of fundamental data items or objects as their elements. The **element type** is specified in angular brackets.

```
// declare a list of int values
std::list<int> my_list();
```

```
// declare a list of std::string objects
std::list<std::string> my_list();
```

- **vector<T>** is a **dynamic array** for type **T** elements, similar to Python lists.
- **deque<T>** ("double ended queue"): **Dynamic array** with capacity both ends.

- **forward_list<T>** is a **singly linked list** data structure for type **T**.
- **list<T>** is a **doubly linked list** data structure for type **T**.

- **set<T>** is a container where each **key** (element) occurs only (at most) once.
- **map<T, V>** contains **key-value** pairs, which each key occurring at most once.
- **multimap<T, V>** contains **key-value** pairs; keys may occur multiple times.

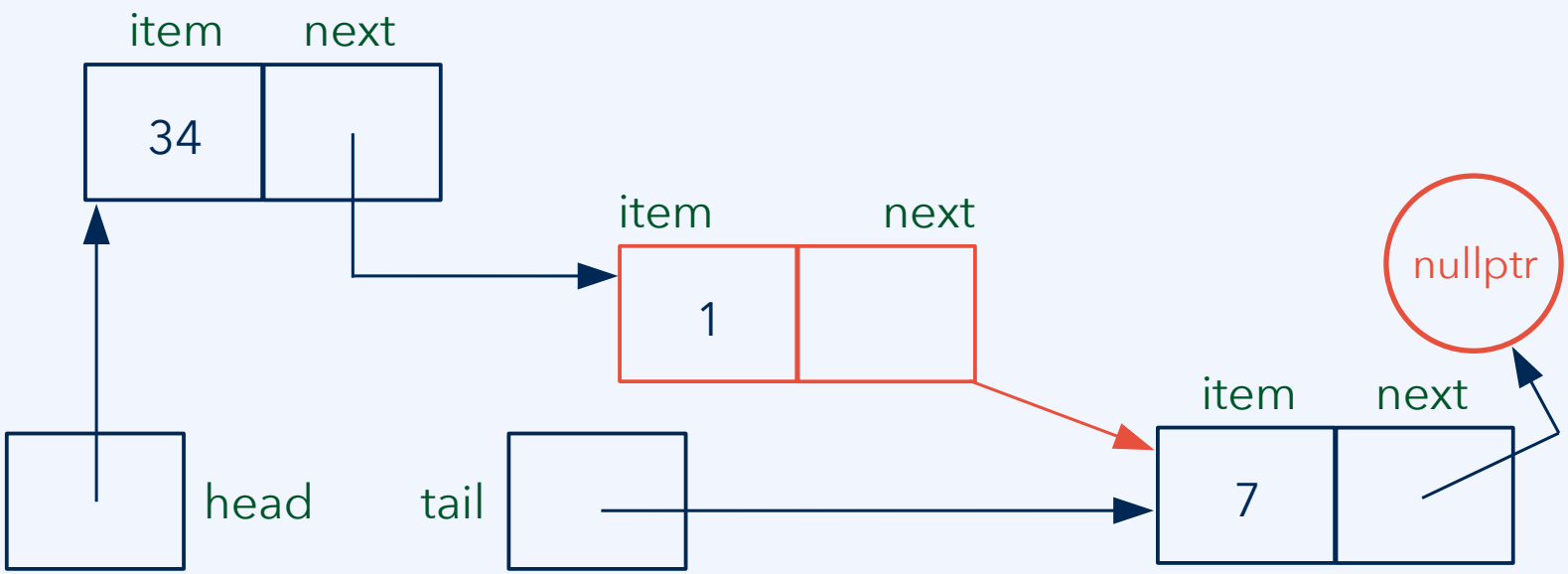
- **array<T, n>** is a **static array** for type **T**, with array size **n**, similar to **T[]** arrays.

Linked lists (singly linked)

Linked lists are dynamic data structures. And they are **linked data structures**.

Their elements are **not contiguous in memory**. Therefore, pointer arithmetics and increments (p++) cannot be used.

Instead, the linked list consists of **nodes**.



Example file: list-template.zip

Templates: Parameterized classes

We have already seen the STL templates: The **same container implementation** can be used for **different types of contained objects**, such as **list<float>** and **list<double>**. We can define our own class templates in this way:

```
template<typename T> class SinglyLinkedListNode
{
public:
  T& get_item() { return this->item; }
  SinglyLinkedListNode<T>* get_next() const { return this->next; }
  void set_item(T in_item) { this->item = in_item; }

private:
  T item;
  SinglyLinkedListNode<T>* next = nullptr;
  void set_next(SinglyLinkedListNode<T>* in_next) { this->next = in_next; }
};
```

attention with split between header and object file; think about "what the compiler will do"

attention with initializations

While there is only one **source code** for each template, **object code** is normally generated separately for each concrete version of it. (But not for the template!)

Example file: list-template.zip

Templates for functions and methods

The same sort of syntax applies for parameterized function and method declarations and definitions. This includes cases with multiple parameters.

```
template<typename T>
  void SinglyLinkedList<T>::push_front(
    const T& pushed_item
) {
  SinglyLinkedListNode<T>* new_node
              = new SinglyLinkedListNode<T>;
  new_node->set_item(pushed_item);

  if(this->empty()) this->tail = new_node;
  else new_node->set_next(this->head);
  this->head = new_node;
}
```

```
template<typename SeqnT, typename ElmnT>
  void test_sequence(
    SeqnT* sqn, int n, int m,
    ElmnT a, ElmnT b, ostream* os
) {
  …
}

template<typename SeqnT, typename ElmnT>
  float test_with_time_measurement(
    SeqnT* sqn, int iterations, ElmnT a, ElmnT b
) {
    int sequence_length = 1000001;
    int deletions = 10;
    test_sequence(sqn, 100000, 10, a, b, &cout);
}
```

Example file: list-template.zip

Templates: Case distinctions

The standard library header <**type_traits**> includes parameterized flags that can be used to make case distinctions, *e.g.*,

- **is_arithmetic**<T>::value, **is_signed**<T>::value, *etc.*;
- **is_pointer**<T>::value, **is_class**<T>::value, **is_array**<T>::value, *etc.*;
- **is_same**<T, S>::value, to check whether T and S are the same type.

In **list-template**, solutions for initializing the property "**T** item" would include:

```
T item = T();
```

```
template<typename T>
    const T initial_value = T();
...
T item = initial_value<T>;
```

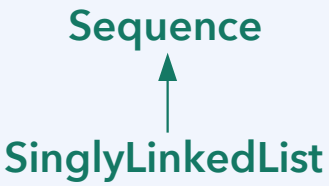
```
SinglyLinkedListNode<T>() {
  if constexpr(is_arithmetic<T>::value) this->item = 3;
  else if constexpr(is_pointer<T>::value) this->item == nullptr;
  else if constexpr(is_same<T, string>::value) this->item = "uninitialized";
  else this->item = T();
}
```

Only with the solution on the right we can make more high-level design distinctions depending on the nature of the type T used for parameterizing.

Abstract classes, concrete subclasses

The code **list-template.zip** has an **abstract class** at the top of a class hierarchy.

Such a class has a **pure virtual** method that is only declared, but not defined. The declaration uses the construction "**virtual** … method(…) **= 0;**".




```
class Sequence
{
public:
  virtual bool empty() const = 0;   // whether sequence is empty
  virtual size_t size() const = 0;  // size (number of items)

  virtual int& front() = 0;  // return reference to first item
  virtual int& back() = 0;   // return reference to final item

  virtual int& at(int i) = 0;  // reference to item at index i

  …
};
```

A class is concrete (*i.e.*, not abstract) if it does *not* have any pure virtual methods.

If it has an abstract superclass, it must **override (define) <u>all</u>** its pure virtual method declarations.

Core guidelines

An abstract class might contain "normal" methods in addition to its pure virtual method(s). If it only has pure virtual methods, it is a pure abstract class. Such classes are used to specify **interfaces**.

- C.120: Use class hierarchies to represent concepts with inherent hierarchical structure (only).
- C.121: If a base class is used as an interface, make it a pure abstract class.
- C.122: Use abstract classes as interfaces when complete separation of interface and implementation is needed.

Concerning virtual methods and overriding:

- C.128: Virtual functions should specify exactly one of virtual, override, or final.

Glossar

Abstrakt klasse

Overstyring

Virtuell metode

Glossar

Abstrakt klasse

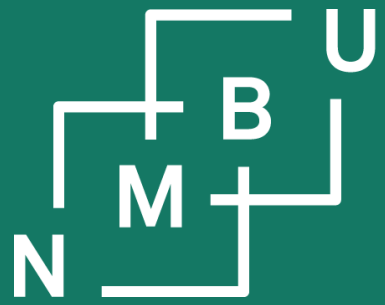
Definisjon: klasse som ikkje er fullstendig definert, og som difor ikkje kan instansierast direkte – berre gjennom subklasser som er konkrete

Overstyring

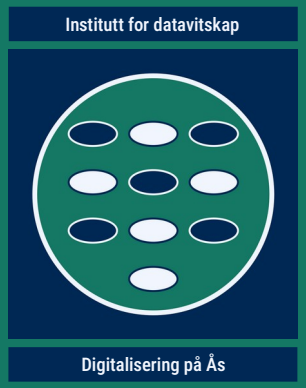
Definisjon: det å overlaste og ta førerang over ein virtuell metode som er deklarert i ei superklasse

Virtuell metode

Definisjon: metode av ein klasse *X* som kan overlastast og overstyrast av ein metode med same namn, definert i ei subklasse *Y* (som er avleidd av *X*), slik at det blir identifisert *ved køyretid* om eit *X*-objekt også er ein instans av klasse *X*



Noregs miljø- og biovitskaplege universitet



INF205
Ressurseffektiv programmering

2 Minnehandtering

2.1 Kallstakken og haugen
2.2 Vanlege minnehandteringsfeil
2.3 Containerklasser (kopiering og flytting)
2.4 Templat og abstrakte klasser